

IHK Lernheft 2025 – Entwicklung & Umsetzung von Algorithmen (FIAE)

Java-ähnlicher Pseudocode (ohne Datentypen) · Jede Sektion enthält: Erklärung · Pseudocode · Beispiel (IHK-Stil) · Tipps & häufige Fehler

1) Zählen, Summieren, Durchschnitt

Worum geht's? Sehr häufige Standardaufgaben: Elemente zählen (z. B. wie viele Werte > Grenzwert), Summe und Durchschnitt berechnen. Benötigt werden eine *Schleife*, eine *Bedingung* und ein *Zähler* bzw. *Akkumulator* (sum).

Pseudocode

```
funktion zaehleGroesser(a, grenze)
  count = 0
  for (i = 0; i < a.length; i++)
    if (a[i] > grenze) count++
  return count
```

```
funktion durchschnitt(a)
  sum = 0
  for (i = 0; i < a.length; i++) sum = sum + a[i]
  return sum / a.length
```

Beispiel (IHK-Stil)

Aufgabe: Gegeben $a = \{5, 7, 2, 9\}$, Grenzwert = 6. Zähle Werte > 6 und berechne den Durchschnitt.

- Zählen: 7 und 9 → **count = 2**
- Summe = $5+7+2+9 = 23$, $n=4$ → **Durchschnitt = $23/4 = 5,75$**

Tipp: Zähler immer mit 0 starten; Durchschnitt erst sum bilden, dann durch `a.length` teilen.

Häufige Fehler: *count* falsch initialisiert, Division durch 0 bei leerem Array (falls möglich), Ganzzahl-/Kommadivision in echten Sprachen.

2) Min / Max (1D-Array)

Worum geht's? Kleinsten bzw. größten Wert ermitteln. Wichtig: *Startwert ist das erste Element* (nicht 0).

Pseudocode

```
funktion findeMin(a)
  min = a[0]
  for (i = 1; i < a.length; i++)
    if (a[i] < min) min = a[i]
  return min
```

```
funktion findeMax(a)
  max = a[0]
  for (i = 1; i < a.length; i++)
    if (a[i] > max) max = a[i]
  return max
```

Beispiel (IHK-Stil)

$a = \{5, 7, 2, 9\} \rightarrow \mathbf{min = 2, max = 9}$

Schrittweise: Start $min=5 \rightarrow 7?$ nein $\rightarrow 2?$ ja $\rightarrow min=2 \rightarrow 9?$ nein.

Tipp: Wenn der *Index* verlangt ist, zusätzlich `minIndex`/`maxIndex` führen.

3) Min / Max (2D-Array)

Worum geht's? 2D-Strukturen (z. B. Monate $\times$ Tage) erfordern *verschachtelte Schleifen*. Häufig wird pro Zeile/Monat eine *Summe* gebildet und daraus Min/Max bestimmt.

Pseudocode (direktes Min)

```
funktion findeMin2D(m)
  min = m[0][0]
  for (i = 0; i < m.length; i++)
    for (j = 0; j < m[i].length; j++)
      if (m[i][j] < min) min = m[i][j]
  return min
```

Pseudocode (Monatssummen & Min/Max)

```
funktion ermittleJahresstatistik(verbrauch) // Monat  $\times$  Tag
  min = MAX_WERT; max = MIN_WERT
  for (m = 0; m < verbrauch.length; m++)
    summe = 0
    for (t = 0; t < verbrauch[m].length; t++)
      summe = summe + verbrauch[m][t]
    if (summe < min) min = summe
    if (summe > max) max = summe
  return new Jahresstatistik(min, max)
```

Beispiel (IHK-Stil)

Aufgabe: Minimaler & maximaler Monatsverbrauch über 12 Monate. Für jede Zeile summieren und vergleichen.

- Monat 1: $\Sigma=120$
- Monat 2: $\Sigma=95 \rightarrow \text{min}=95$
- ...
- Monat 8: $\Sigma=180 \rightarrow \text{max}=180$

Tipp: Äußere Schleife = Zeilen/Monate, innere Schleife = Spalten/Tage.

4) Objektrückgabe (z. B. Jahresstatistik)

Worum geht's? Statt zwei Zahlen getrennt zurückzugeben, wird eine *Klasse* genutzt und ein *Objekt* zurückgegeben. Das ist in AP2 sehr beliebt.

Klasse & Nutzung

```
klasse Jahresstatistik(min, max)
// Attribute: minVerbrauch, maxVerbrauch
// Konstruktor setzt Werte
// Getter: getMinVerbrauch(), getMaxVerbrauch()
```

```
funktion ermittleJahresstatistik(a)
// ... min/max berechnen ...
return new Jahresstatistik(min, max)
```

Beispiel (IHK-Stil)

Energie-Verbrauch (Monatssummen) → gib Objekt mit `min` und `max` zurück und nutze Getter.

Tipp: In der Lösung `new Klassenname(...)` explizit angeben.

● 5) If/Else + Counter („besser als DAX“)

Worum geht's? Pro Eintrag prüfen und Zähler erhöhen. In Aktien-Aufgaben wird die prozentuale Veränderung verglichen.

Pseudocode

```
funktion notierungPlus(kurse)
  count = 0
  for (i = 0; i < kurse.length; i++)
    if (kurse[i].aktieProzent > kurse[i].daxProzent) count++
  return count
```

Beispiel (IHK-Stil)

Tage: (Aktie, DAX) = (1.2,0.8), (-0.5,0.2), (0.3,0.1) → **2** Tage „besser“.

Häufige Fehler: count++ vergessen; falscher Operator; Rückgabe innerhalb der Schleife.

● 6) BubbleSort

Worum geht's? Wiederholt benachbarte Elemente vergleichen und tauschen. Nach Runde i steht das größte der restlichen Elemente am Ende.

Pseudocode

```
funktion bubbleSort(a)
  for (i = 0; i < a.length - 1; i++)
    for (j = 0; j < a.length - 1 - i; j++)
      if (a[j] > a[j+1]) swap(a[j], a[j+1])
```

Beispiel (IHK-Stil)

$a = \{5, 2, 4\} \rightarrow \{2, 4, 5\}$ (siehe Schrittfolge)

Tipp: Innere Grenze $n-1-i$ beachten; optional „early exit“, wenn keine Tausche in einer Runde.

7) InsertionSort

Worum geht's? Einsortieren in den linken (schon sortierten) Teil. Stabil, gut für fast sortierte Eingaben.

Pseudocode

```
funktion insertionSort(a)
  for (i = 1; i < a.length; i++)
    key = a[i]
    j = i - 1
    while (j >= 0 && a[j] > key)
      a[j+1] = a[j]
      j = j - 1
    a[j+1] = key
```

Beispiel (IHK-Stil)

$a = \{5, 2, 4\} \rightarrow$ nach zwei Iterationen $\{2,4,5\}$ (Schieben + Einfügen).

Häufige Fehler: $a[j+1]=key$ vergessen; j falsch dekrementiert.

● 8) Lineare Suche

Worum geht's? Von links nach rechts prüfen. Rückgabe Index oder -1.

Pseudocode

```
funktion linearSearch(a, x)
  for (i = 0; i < a.length; i++)
    if (a[i] == x) return i
  return -1
```

Beispiel (IHK-Stil)

$a=\{5,7,2\}$, $x=7 \rightarrow 1$; $x=8 \rightarrow -1$.

Tipp: Wenn mehrere Vorkommen $\rightarrow$ häufig reicht das erste; sonst zusätzlich zählen.

● 9) Objektvergleich mit Function vergleiche

Worum geht's? Sortieren nach flexiblen Kriterien (Name, Datum, Prozent). Die Vergleichsfunktion kapselt die Logik.

Pseudocode

```
funktion sort(a, vergleiche)
  for (i = 0; i < a.length - 1; i++)
    for (j = 0; j < a.length - 1 - i; j++)
      if (vergleiche(a[j], a[j+1]) > 0) swap(a[j], a[j+1])
```

Beispiel (IHK-Stil)

Tageskurse nach Datum sortieren: `vergleiche(x,y)` gibt `>0` zurück, wenn x nach y.

Tipp: Bei Objekten nie direkt `x > y`; immer `vergleiche(x,y)`!

● 10) BinarySearch (selten)

Worum geht's? Sucht in sortierten Feldern sehr effizient.

Pseudocode

```
funktion binarySearch(a, x)
  left = 0; right = a.length - 1
  while (left <= right)
    mid = (left + right) / 2
    if (a[mid] == x) return mid
    if (a[mid] < x) left = mid + 1
    else right = mid - 1
  return -1
```

Beispiel (IHK-Stil)

$a=\{2,4,7,9\}$, $x=7 \rightarrow$ Index 2.

Häufige Fehler: Unsortiertes Array; left/right nicht korrekt angepasst.

● 11) Rekursion (Fibonacci)

Worum geht's? Theorie-Thema; iterative Varianten sind praxisnäher.

Pseudocode

```
funktion fib(n)
  if (n <= 1) return 1
  else return fib(n-1) + fib(n-2)
```

Beispiel (IHK-Stil)

fib(4) = 5 (Baumdarstellung möglich).

● 12) QuickSort / MergeSort (Theorie)

- **QuickSort:** Pivot wählen, partitionieren, rekursiv sortieren. Ø $O(n \log n)$, Worst Case $O(n^2)$.
- **MergeSort:** Teilen, rekursiv sortieren, zusammenführen. Immer $O(n \log n)$.

Hinweis: In AP2 meist nur vom Prinzip her gefragt.

Druck: Browser → Drucken → Als PDF speichern.